

< 실습 7 >

- 실습과제 1

```
void *consumer(void *arg) {
    int i;
    for( i=0; i<10; i++) {
        sem_wait( &full );
        sem_wait( &mutex );
        printf("Consumer Read: %2d from %2d-th buffer \n", buffer[i%4], i%4);
        buffer[ i%4 ] = -1;
        sem_post( &mutex );
        sleep(2);
    }
}

void *producer(void *arg) {
    int i;
    for( i=0; i<10; i++) {
        sem_wait( &mutex );
        printf("Producer Write: %2d into %2d-th buffer \n", i, i%4);
        buffer[ i%4 ] = i;
        sem_post( &mutex );
        sem_post( &full );
        sleep(1);
    }
}

int main(void) {
    pthread_t id_p1, id_p2;

    printf("Thread, made by WanLee \n");
    sem_init( &mutex, 0, 1 );
    sem_init( &full, 0, 0 );
}
```

위의 코드는 전체 코드 중에서, 주요 부분 코드이다. 이 코드는 buffer 배열에 producer() 함수가 실행되어 저장된 개수에서, consumer() 함수가 실행되어 삭제되고 남은 저장 개수를 full 세마포어로 계산하고, full 세마포 변수 값이 0이면 consumer() 함수를 대기하게 동기화 시킨다. 이 코드의 실행 결과는 아래와 같다.

```
[wanlee@CentOS32 work]$ ./a.out
Thread, made by WanLee
Producer Write: 0 into 0-th buffer
Consumer Read: 0 from 0-th buffer
Producer Write: 1 into 1-th buffer
Consumer Read: 1 from 1-th buffer
Producer Write: 2 into 2-th buffer
Producer Write: 3 into 3-th buffer
Consumer Read: 2 from 2-th buffer
Producer Write: 4 into 0-th buffer
Producer Write: 5 into 1-th buffer
Consumer Read: 3 from 3-th buffer
Producer Write: 6 into 2-th buffer
Producer Write: 7 into 3-th buffer
Consumer Read: 4 from 0-th buffer
Producer Write: 8 into 0-th buffer
Producer Write: 9 into 1-th buffer
Consumer Read: 9 from 1-th buffer
Consumer Read: 6 from 2-th buffer
Consumer Read: 7 from 3-th buffer
Consumer Read: 8 from 0-th buffer
Consumer Read: -1 from 1-th buffer
[wanlee@CentOS32 work]$
```

```

void *consumer(void *arg) {
    int i;
    for( i=0; i<10; i++) {
        sem_wait( &full );
        sem_wait( &mutex );
        printf("Consumer Read: %2d from %2d-th buffer \n", buffer[i%4], i%4);
        buffer[ i%4 ] = -1;
        sem_post( &mutex );
        sem_post( &empty );
        sleep(2);
    }
}

void *producer(void *arg) {
    int i;
    for( i=0; i<10; i++) {
        sem_wait( &empty );
        sem_wait( &mutex );
        printf("Producer Write: %2d into %2d-th buffer \n", i, i%4);
        buffer[ i%4 ] = i;
        sem_post( &mutex );
        sem_post( &full );
        sleep(1);
    }
}

int main(void) {
    pthread_t id_pl, id_p2;

    printf("Thread, made by Wanlee \n");
    sem_init( &mutex, 0, 1 );
    sem_init( &full, 0, 0 );
    sem_init( &empty, 0, 4 );
}

```

위의 코드는 전체 코드 중에서, 주요 부분 코드이다. 이 코드는 buffer 배열에 consumer() 함수가 삭제하여 사용가능한 공간 개수에서, producer() 함수가 실행되어 사용할수 없게 된 공간 개수를 empty 세마포어로 계산하고, empty 세마포 변수 값이 0이면 producer() 함수를 대기하게 동기화 시킨다. 실행 결과는 실습 안내 ppt 자료 7페이지에서 확인할수 있다.

- 실습과제 2 (선택 과제)

consumer() 함수와 producer() 함수의 sem_wait() 동작 순서를 일치시킨다.

예, consumer() 함수와 producer() 함수 코드에서 동일한 순서로 mutex1과 mutex2를 사용

```
sem_wait( &mutex1 );
```

```
sem_wait( &mutex2 );
```

수정 이전 코드에서는 consumer() 함수에서 mutex1의 값을 0으로 변경하고, 이후에 sleep(1)로 인해서 producer() 함수가 실행되면서 mutex2의 값을 0으로 변경시키게 된다. 그러면 consumer() 함수는 sem_wait(&mutex2) 부분에서 무한대기하고, producer() 함수는 sem_wait(&mutex1) 부분에서 무한대기하게 된다.

수정 이후에는 consumer() 함수에서 mutex1의 값을 0으로 변경하고, 이후에 sleep(1)로 인해서 producer() 함수가 실행되더라도 sem_wait(&mutex1) 부분에서 대기하게 되어 mutex2의 값을 0으로 변경하지 못한다. 따라서 consumer() 함수는 sem_wait(&mutex2) 부분에서 무한대기하고 producer() 함수는 sem_wait(&mutex1) 부분에서 무한대기하는 현상은 발생하지 않게 된다.

< 실습 8 >

- 실습과제 1

```
drwxr-xr-x. 2 wanlee wanlee 4096 2019-10-20 06:07 test1
drwxr-xr-x. 2 wanlee wanlee 4096 2019-10-20 06:07 test2
drwxr-xr-x. 2 wanlee wanlee 4096 2019-12-13 07:08 work
drwxr-xr-x. 2 wanlee wanlee 4096 2017-03-03 02:46 공개
drwxr-xr-x. 2 wanlee wanlee 4096 2017-03-03 02:46 다운로드
drwxr-xr-x. 2 wanlee wanlee 4096 2017-03-03 02:46 문서
drwxr-xr-x. 2 wanlee wanlee 4096 2017-03-03 02:46 바탕화면
drwxr-xr-x. 2 wanlee wanlee 4096 2017-03-03 02:46 비디오
drwxr-xr-x. 2 wanlee wanlee 4096 2017-03-03 02:46 사진
drwxr-xr-x. 2 wanlee wanlee 4096 2017-03-03 02:46 음악
drwxr-xr-x. 2 wanlee wanlee 4096 2017-03-03 02:46 템플릿
[wanlee@CentOS32 ~]$ chmod u=rwx,go=rw /home/wanlee/test1
[wanlee@CentOS32 ~]$ ls -l
합계 60
drwxrwxr-x. 3 wanlee wanlee 4096 2019-10-20 05:56 20110001
-rw-rw-r--. 1 wanlee wanlee 24 2019-10-20 05:58 sample
-rw-rw-r--. 1 wanlee wanlee 40 2019-10-20 06:04 sample2
-rw-rw-r--. 1 wanlee wanlee 20 2019-10-20 06:04 sample3
drwxrw-rw-. 2 wanlee wanlee 4096 2019-10-20 06:07 test1
drwxr-xr-x. 2 wanlee wanlee 4096 2019-10-20 06:07 test2
```

```
[wanlee@CentOS32 ~]$ chmod 766 ./test2
[wanlee@CentOS32 ~]$ ls -l
합계 60
drwxrwxr-x. 3 wanlee wanlee 4096 2019-10-20 05:56 20110001
-rw-rw-r--. 1 wanlee wanlee 24 2019-10-20 05:58 sample
-rw-rw-r--. 1 wanlee wanlee 40 2019-10-20 06:04 sample2
-rw-rw-r--. 1 wanlee wanlee 20 2019-10-20 06:04 sample3
drwxrw-rw-. 2 wanlee wanlee 4096 2019-10-20 06:07 test1
drwxrw-rw-. 2 wanlee wanlee 4096 2019-10-20 06:07 test2
drwxr-xr-x. 2 wanlee wanlee 4096 2019-12-13 07:08 work
```

```
[wanlee@CentOS32 ~]$ cd ..
[wanlee@CentOS32 home]$ pwd
/home
[wanlee@CentOS32 home]$ ls -al
합계 12
drwxr-xr-x. 3 root root 4096 2017-03-03 02:46 .
dr-xr-xr-x. 24 root root 4096 2019-12-13 05:03 ..
drwxrw-rw-. 31 wanlee wanlee 4096 2019-12-13 07:08 wanlee
[wanlee@CentOS32 home]$ chmod 700 .
chmod: changing permissions of `.`: 명령을 허용하지 않음
[wanlee@CentOS32 home]$
```

/home 디렉토리의 소유자는 root이므로, root가 아닌 일반사용자가 이 디렉토리의 권한을 변경할수 없음.

- 실습과제 2

```
[wanlee@CentOS32 ~]$ pwd
/home/wanlee
[wanlee@CentOS32 ~]$ cd ..
[wanlee@CentOS32 home]$ pwd
/home
[wanlee@CentOS32 home]$ tar cf ./wanlee/20110901.tar ./wanlee
tar: ./wanlee/20110901.tar: file is the archive; not dumped
[wanlee@CentOS32 home]$ cd wanlee
[wanlee@CentOS32 ~]$ ls
20110001      sample  sample3  test1  공개  문서  비디오  음악
20110901. tar sample2  test2  work  다운로드  바탕화면  사진  템플릿
[wanlee@CentOS32 ~]$ mkdir backup
[wanlee@CentOS32 ~]$ mv 20110901.tar backup
[wanlee@CentOS32 ~]$ cd backup
[wanlee@CentOS32 backup]$ tar xf 20110901.tar
[wanlee@CentOS32 backup]$ ls
20110901. tar  wanlee
[wanlee@CentOS32 backup]$ █
```

```
[wanlee@CentOS32 backup]$ cd /home
[wanlee@CentOS32 home]$ tar cfz ./wanlee/20110902.tar.gz ./wanlee
tar: ./wanlee: file changed as we read it
[wanlee@CentOS32 home]$ cd wanlee
[wanlee@CentOS32 ~]$ ls
20110001      backup  sample2  test1  work  다운로드  바탕화면  사진  템플릿
20110902. tar.gz sample  sample3  test2  공개  문서  비디오  음악
[wanlee@CentOS32 ~]$ mkdir backup2
[wanlee@CentOS32 ~]$ mv 20110902.tar.gz backup2
[wanlee@CentOS32 ~]$ cd backup2
[wanlee@CentOS32 backup2]$ tar xfz 20110902.tar.gz
[wanlee@CentOS32 backup2]$ ls
20110902. tar.gz  wanlee
[wanlee@CentOS32 backup2]$ █
```

```
[root@CentOS32 home]# chmod 766 .
[root@CentOS32 home]# ls -al
합계 12
drwxrw-rw-. 3 root root 4096 2017-03-03 02:46
dr-xr-xr-x. 24 root root 4096 2019-12-13 05:03
drwxrw-rw-. 31 wanlee wanlee 4096 2019-12-13 07:26
[root@CentOS32 home]#
```

```
[root@CentOS32 home]# chmod 755 .
[root@CentOS32 home]# ls -al
합계 12
drwxr-xr-x. 3 root root 4096 2017-03-03 02:46 .
dr-xr-xr-x. 24 root root 4096 2019-12-13 05:03 ..
drwxrw-rw-. 31 wanlee wanlee 4096 2019-12-13 07:26 .
[root@CentOS32 home]#
```

/home 디렉토리가 root가 아닌 기타 사용자에게는 읽기와 실행 권한을 허용하여, ls 명령어와 vi 명령어 그리고 cd 명령어의 접근을 허용한다.

< 실습 9 >

- 실습과제 1

생산자 프로그램과 소비자 프로그램이 하나의 파일을 동시에 접근하여 일부가 쓰기를 실행하면 임계영역을 해치므로 잘못된 결과가 출력된다. (동시에 접근하여 모두 읽기를 실행하면 문제 발생하지 않음)

소비자 프로그램이 파일을 open 할 때마다 항상 파일의 맨 앞부터 읽으므로 0, 1, 2, 3, 4를 반복해서 읽음. 5 이상의 값을 읽기 위해서는 파일 앞부분의 5줄을 skip하고 나서 읽거나, 또는 파일 앞부분의 5줄을 삭제하는 과정이 필요함.

실습 안내 ppt 파일의 7페이지 프로그램은 fseek() 함수를 사용하여 파일 앞부분의 5줄을 이동하는 과정을 수행함. 5줄을 이동한 이후에 읽으면 5, 6, 7, 8, 9의 값을 읽을수 있음. 실행 결과는 아래와 같음.

```
[wanlee@CentOS32 lab09]$ ./consume
File Comm., made by Wanlee
Line length: 22
Producer increase: 5
Producer increase: 6
Producer increase: 7
Producer increase: 8
Producer increase: 9
[wanlee@CentOS32 lab09]$
```

< 실습 10 >

```
#include <stdio.h>
#include <fcntl.h>

int main(void) {
    int    fd1, fd2;
    int    n_write, n_read, msg, i;

    mkfifo("/tmp/my_pipe1", 0644);

    if( (fd1 = open("/tmp/my_pipe1", O_WRONLY, 0644)) == -1 )
    {
        printf("Pipe 1 Open Error \n"); return -1; }

    if( (fd2 = open("/tmp/my_pipe2", O_RDONLY, 0644)) == -1 )
    {
        printf("Pipe 2 Open Error \n"); return -1; }

    for( i=0; i<10; i++ )
    {
        if( (n_write = write(fd1, &i, sizeof(i))) == -1 )
        {
            printf("Write Error \n"); return -1; }
        printf("Producer Sent: %d,", i);

        if( (n_read = read(fd2, &msg, sizeof(msg))) == -1 )
        {
            printf("Read Error \n"); return -1; }
        printf("Producer Received: %d \n", msg);
        sleep(1);
    }

    close( fd1 ); close( fd2 );
    return 0;
}

"new_producer.c" 31L, 715C 31, 0-1
```

```
#include <stdio.h>
#include <fcntl.h>

int main(void) {
    int    fd1, fd2;
    int    n_write, n_read, msg, i;

    mkfifo("/tmp/my_pipe2", 0644);

    if( (fd1 = open("/tmp/my_pipe1", O_RDONLY, 0644)) == -1 )
    {
        printf("Pipe 1 Open Error \n"); return -1; }

    if( (fd2 = open("/tmp/my_pipe2", O_WRONLY, 0644)) == -1 )
    {
        printf("Pipe 2 Open Error \n"); return -1; }

    for( i=0; i<10; i++ )
    {
        if( (n_read = read(fd1, &msg, sizeof(msg))) == -1 )
        {
            printf("Read Error \n"); return -1; }
        printf("Consumer Received: %d,", msg);
        msg = msg + 10;

        if( (n_write = write(fd2, &msg, sizeof(msg))) == -1 )
        {
            printf("Write Error \n"); return -1; }
        printf("Consumer Sent: %d \n", msg);

        sleep(2);
    }

    close( fd1 ); close( fd2 );
    return 0;
}

"new_consumer.c" 33L, 740C 저장 했습니다 3, 0-1
```

< 실습 11 >

- 실습과제 1

아래는 수정된 주요 코드 부분만을 보여주고 있다.

```
    for( i=0; i<10; i++ ) {
        send( sock2, &i, sizeof(i), 0 );
        printf("Server Send: %d \n", i);

        recv( sock2, &msg, sizeof(msg), 0 );
        printf("Server Received: %d \n", msg);
        sleep(1);
    }

    close(sock2); close(sock1);
    return 0;
}
"server.c" 33L, 923C 저장 했습니다
```

```
    printf("Client: Connection Established. \n");
    for( i=0; i<10; i++ ) {
        if( recv(sock, &read, sizeof(read), 0) == -1 )
        {
            printf("Client: Receive Error \n"); return -1; }
        printf("Client Received: %d \n", read);

        read = read + 10;
        send( sock, &read, sizeof(read), 0 );
        printf("Client Send: %d \n", read);
        sleep(2);
    }

    close(sock);
    return 0;
}
"client.c" 32L, 813C 저장 했습니다
```

32, 2-8

- 실습과제 2 (선택 과제)

```
int main() {
    int sock1, sock2, sock3, length, i, msg;
    struct sockaddr_un server, client;

    if( (sock1 = socket(AF_UNIX, SOCK_STREAM, 0)) == -1 )
    {
        printf("Server: Open Error \n"); return -1; }
    server.sun_family = AF_UNIX;
    strcpy( server.sun_path, "/tmp/sock_box" );

    if( bind(sock1, (struct sockaddr *)&server, sizeof(server)) == -1 )
    {
        printf("Server: Bind Error \n"); return -1; }
    if( listen(sock1, 5) == -1 )
    {
        printf("Server: Listen Error \n"); return -1; }

    if( (sock2 = accept(sock1, (struct sockaddr *)&client, &length)) == -1 )
    {
        printf("Server: Accept 2 Error \n"); return -1; }

    if( (sock3 = accept(sock1, (struct sockaddr *)&client, &length)) == -1 )
    {
        printf("Server: Accept 3 Error \n"); return -1; }

    for( i=0; i<10; i++ ) {
        send( sock2, &i, sizeof(i), 0 );
        printf("Server Send: %d \n", i);

        recv( sock3, &msg, sizeof(msg), 0 );
        printf("Server Received: %d \n", msg);
        sleep(1);
    }

    close(sock2); close(sock3); close(sock1);
    return 0;
}
```

"new_server.c" 37L, 1077C 저장 했습니다

37, 2-8

```
int main() {
    int sock2, sock3, i, read;
    struct sockaddr_un client;

    if( (sock2 = socket(AF_UNIX, SOCK_STREAM, 0)) == -1 )
    {
        printf("Client: Open Error \n"); return -1; }
    if( (sock3 = socket(AF_UNIX, SOCK_STREAM, 0)) == -1 )
    {
        printf("Client: Open Error \n"); return -1; }

    client.sun_family = AF_UNIX;
    strcpy( client.sun_path, "/tmp/sock_box");

    if( connect(sock2, (struct sockaddr *)&client, sizeof(client)) == -1 )
    {
        printf("Client: Conn. 2 Error \n"); return -1; }

    if( connect(sock3, (struct sockaddr *)&client, sizeof(client)) == -1 )
    {
        printf("Client: Conn. 3 Error \n"); return -1; }

    printf("Client: Connection Established. \n");
    for( i=0; i<10; i++ ) {
        if( recv(sock2, &read, sizeof(read), 0) == -1 )
        {
            printf("Client: Receive Error \n"); return -1; }
        printf("Client Received: %d \n", read);

        read = read + 10;
        send( sock3, &read, sizeof(read), 0 );
        printf("Client Send: %d \n", read);
        sleep(2);
    }

    close(sock2); close(sock3);
    return 0;
}
```

"new_client.c" 39L, 1075C 저장 했습니다

39, 0-1